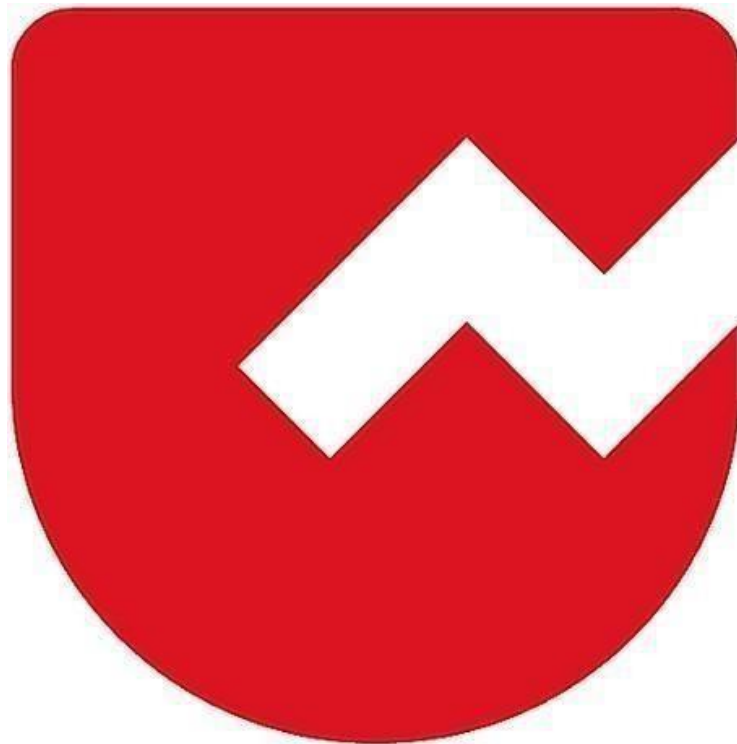




# **Zoho Schools for Graduate Studies**



**Notes**

## Static

- It belongs to class and it have class level access.
- Values / data doesn't change from object to object , it should be declare with static keyword.
- It can be accessed anywhere on the block or class
- If we want to access access outside the class,

**ClassName.StaticMember**

### When to you use static ?

- Common properties for all objects.
- To store single copy in a class.

### Real Time Example for static usage :

- DB Connections
- Network Sockets

### Static Block :

- Static blocks will execute whenever the class gets loaded for the very first time in the memory .
- Static block was executed before main method and object creation during class loading .
- If there are multiple static blocks present, they will execute in which the order they have present
- Static blocks doesn't need object creation

### Can a constructor be static ?

No , It is object specific , we can't make it as static.

## Non – static Block :

- It is also called as instance initializer.
- It runs prior to the constructor ie) it runs before the constructor and after the object creation.

## When to use Instance Initializer ?

Instance Initializer initializes instance variables before the constructor runs

| Feature         | Static Block                          | Non-Static Block (Instance Initializer)         |
|-----------------|---------------------------------------|-------------------------------------------------|
| When it runs    | When the <b>class is loaded</b>       | When an <b>object is created</b>                |
| Number of times | <b>Once only</b>                      | <b>Every time</b> an object is created          |
| Purpose         | Initialize <b>static variables</b>    | Initialize <b>instance variables</b>            |
| Syntax          | <code>static { }</code>               | <code>{ }</code>                                |
| Executes before | Before any object creation            | Before constructor                              |
| Access          | Can access <b>only static members</b> | Can access <b>static and non-static members</b> |
| Belongs to      | <b>Class</b>                          | <b>Object</b>                                   |
| Use case        | Load configuration, static data       | Common initialization for all constructors      |

## Code Snippets :

1) package part1;

```
public class StaticDemo {
```

```
public static void main(String[] args) {  
    System.out.println("Inside main method !");  
}  
  
static {  
    System.out.println("Inside static block!");  
}  
}
```

**Output :**

Inside static block!

Inside main method !

**Explanation :**

Static block executes first because it runs during class loading, before the main method is executed.

2) package part1;

```
public class StaticDemo {  
  
    public static void main(String[] args) {  
        System.out.println("Inside main method !");  
    }  
  
    static {
```

```
        System.out.println("Inside static block 1!");
    }

    static void m() {
        System.out.println("Inside static method");
    }

    static {
        System.out.println("Inside static block 2!");
        m();
    }
}
```

**Output:**

Inside static block 1!  
Inside static block 2!  
Inside static method  
Inside main method !

**Explanation :**

Static blocks execute first during class loading in top-to-bottom order, and only after that the main method runs.

3) package part1;

```
public class StaticDemo {

    StaticDemo() {
        System.out.println("Inside main method !");
    }

    public static void main(String[] args) {
        System.out.println("Inside main method !");
    }

    static {
        System.out.println("Inside static block 1!");
    }

    static void m() {
        System.out.println("Inside static method");
    }

    static {
        System.out.println("Inside static block 2!");
        // m();
    }
}
```

**Output :**

Inside static block 1!

Inside static block 2!

Inside main method !

**Explanation :**

Static blocks execute during class loading, constructor executes only when an object is created, and main method runs after static blocks.

4) package part1;

```
public class StaticDemo {
```

```
    StaticDemo() {
```

```
        System.out.println("Inside constructor !");
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        new StaticDemo();
```

```
        System.out.println("Inside main method !");
```

```
    }
```

```
    static {
```

```
        System.out.println("Inside static block 1!");
```

```
    }
```

```
static void m() {  
    System.out.println("Inside static method");  
}  
  
static {  
    System.out.println("Inside static block 2!");  
}  
}
```

**Output :**

Inside static block 1!  
Inside static block 2!  
Inside constructor !  
Inside main method !

**Explanation :**

Static blocks run first when the class loads, constructor runs when an object is created inside main, and then the remaining main code executes.

5) package part1;

```
public class StaticDemo {
```



```
// Non-static block 1 (Instance Initializer)
{
    System.out.println("Inside Non-static block 1!");
}
```

```
// Non-static block 2 (Instance Initializer)
{
    System.out.println("Inside Non-static block 2!");
}
```

```
// Constructor
StaticDemo() {
    System.out.println("Inside constructor");
}
```

```
public static void main(String[] args) {
    System.out.println("Inside main method !");
    new StaticDemo();
}
```

```
// Static block 1
static {
    System.out.println("Inside static block 1!");
}
```

```
// Static method (not called)
static void m() {
    System.out.println("Inside static method");
}

// Static block 2
static {
    System.out.println("Inside static block 2!");
}
}
```

### **OutPut :**

Inside static block 1!

Inside static block 2!

Inside main method !

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

### **Explanation :**

Static blocks execute during class loading, non-static blocks execute during object creation before the constructor, and main runs after static blocks.

6) package part1;

```
public class StaticDemo {

    // Non-static block 1
    {
        System.out.println("Inside Non-static block 1!");
    }

    // Non-static block 2
    {
        System.out.println("Inside Non-static block 2!");
    }

    // Constructor
    StaticDemo() {
        System.out.println("Inside constructor");
    }

    public static void main(String[] args) {
        System.out.println("Inside main method !");
        new StaticDemo(); // first object
        new StaticDemo(); // second object
    }
}
```

```
// Static block 1
static {
    System.out.println("Inside static block 1!");
}

// Static method (not called)
static void m() {
    System.out.println("Inside static method");
}

// Static block 2
static {
    System.out.println("Inside static block 2!");
}
}
```

### **Output :**

Inside static block 1!

Inside static block 2!

Inside main method !

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

Inside Non-static block 1!

Inside Non-static block 2!

Inside constructor

**Explanation :**

Static blocks execute once during class loading, non-static blocks and constructors execute every time an object is created.